A I P O C K E T A G E N C Y

aipocketagency.com

[apa · kit 02]

The Dev-Team Document Set

Eleven templates and three conventions that turn a solo build into a dev team that documents itself. The artifacts other operators sell for $497, with the audit layer that keeps them honest.

KIT	PRICE	FORMAT
02 / 05	$15 USD	10 pages

Chase Whited · 2026-05-12

TABLE OF CONTENTS

SECTION 1.

Why solo operators ship slop without this

A real dev team has rhythm — decisions get logged, conventions get written down, somebody is responsible for keeping the docs current. Solo operators with Claude Code skip all of it and call it speed. The slop shows up six weeks later.

I shipped my first solo build with Claude Code and zero documentation discipline. I had a CLAUDE.md, half-written. I had no decision log, no ADRs, no change log. I had memory files but no convention for when to add to them and when to leave them alone. Six weeks in, the agent started contradicting itself across sessions — it would suggest one thing on Tuesday and the opposite on Thursday, both with confidence, because the only source of truth was whatever was in the current context window. The codebase grew faster than my ability to track why anything was the way it was.

That's the wall this kit was built against. A solo operator with an agent is not a team of one — it is structurally a team of three or four, except none of the team members read minutes. The CLAUDE.md agent in Cursor doesn't remember what the Claude Code agent did last week. The Codex agent doesn't know which convention you locked in for Stripe integration. Without artifacts, every session relitigates choices that were already settled. The slop is the cost of that relitigation, paid in code.

The fix is dev-team discipline applied to a solo operator. You pretend you have a five-person team and you document for that team. Every decision gets a row in the decision log. Every shipped change gets a line in the change log. Every convention is in CLAUDE.md or in an ADR. The agents read the artifacts at session start and inherit the past instead of guessing at it. The operator's job moves from "remember everything" to "keep the artifacts current," which is a far smaller job than the one that breaks people.

This kit is that artifact stack — eleven document templates, ready to drop into any repo, plus the three conventions that turn the stack from a static snapshot into something that evolves cleanly. Other operators on the AI workflow scene sell exactly this artifact set for $497. The kit is $15 because the artifacts are the easy half. The community at `aipocketagency.com` is the live system that keeps them current as Claude, Cursor, and Codex all ship breaking changes. The kit advertises the community; the community is where the artifacts stay alive.

> WHAT THE KIT IS

Eleven document templates plus three operational conventions, dropped into a `templates/` folder you copy into your repo. Each template ships with example entries, not just headers. You fill in your project specifics, commit, point your agents at the repo, and the agents inherit the conventions on day one.

SECTION 2.

How to use the bundle (and the order)

Drop the templates into your repo, fill them out in the order below, and tell your agent to read them at session start. The order matters — each doc consumes the one above it.

The bundle ships as a flat folder structure you copy into your project. The convention is to put everything under `docs/` at repo root so the agent finds it without hunting. CLAUDE.md sits at the project root (Claude Code looks for it there). AGENTS.md sits at the project root too (the cross-agent convention that Codex, Cursor, and others respect). Everything else nests under `docs/`.

REPO ROOT	`CLAUDE.md`, `AGENTS.md`, `README.md`
DOCS/	`MEMORY.md`, `DECISIONS.md`, `CHANGELOG.md`, `FEATURES.md`
DOCS/ CONVENTIONS/	`CODING.md`, `SECURITY.md`, `DEPLOYMENT.md`
DOCS/ADRS/	`0001-stack-choice.md` (plus future ADRs)
DOCS/ CONVENTIONS/	the three operational conventions (see §4)

The fill order is not arbitrary. Each doc consumes context from the ones above it, so building bottom-up creates rewrites you don't need:

1. **CLAUDE.md first.** Project overview, stack, conventions reference, agent rules. Two pages, max. The agent reads this before anything else.
2. **AGENTS.md second.** Agent-specific rules — what every assistant should always do, what it should never do, the safety rails. Shorter than CLAUDE.md.
3. **MEMORY.md scaffold third.** The 4-type pattern (`user` , `feedback` , `project` , `reference`) plus an index. You start with zero entries and build up over time. Don't try to pre-populate.
4. **Decision Log fourth.** This is the document you'll come back to most. Every meaningful technical decision gets a row.
5. **Change Log, Feature Inventory, conventions docs, ADRs** — fill these as the project demands them. Don't pre-fill; backfill from real work.

Five docs filled out in the first hour is the day-one bar. The other six get backfilled as the project asks for them. Trying to fill all eleven from scratch is the failure mode — it produces fiction the agent then treats as fact.

SECTION 3.

The eleven document templates

Eleven templates, each with example entries you can imitate. The point isn't to use every section — it's to have the section ready when you need it.

1. CLAUDE.md — master context

The annotated CLAUDE.md template covers project context, stack overview, code conventions (with link to the conventions doc as canonical), agent rules, gotchas, "what to read first" (ordered file list for agent startup), and "what to never touch" (files the agent must not modify without approval). Two pages max. The agent reads this first.

2. AGENTS.md — system-wide agent behavior

The cross-agent rulebook. Startup behavior (rebase on main first), safety rules (no destructive commands without confirmation), agent coordination conventions when multiple agents work in parallel. Half a page, max. Every modern agent respects it.

3. MEMORY.md — the 4-type pattern

The scaffold for file-based memory. Four memory types — `user` (who is the operator), `feedback` (guidance about how to work), `project` (in-flight context that decays), `reference` (pointers to external systems). Each type has its own entry format. MEMORY.md is the index; the actual memories live in their own files. Example entries for each type ship in the kit.

4. Decision Log — `DECISIONS.md`

The technical decision row format: date, decision, alternatives considered, rationale, dependencies, status. Three example entries ship pre-filled — one stack decision, one architecture decision, one process decision. The Decision Log is the doc you'll thank yourself most for keeping current, because it's how you stop relitigating the same choice three times.

5. Change Log — `CHANGELOG.md`

The shipped-change ledger. Format: date, change, owner, why, link to commit. Three example entries pre-filled. The Change Log is the doc that lets a returning agent skim the last 30 days of work without reading commits. Don't try to capture every commit — capture the ones that change the system's behavior.

6. Feature Inventory — `FEATURES.md`

The feature-by-feature snapshot. Format: feature name, status (`live` / `wip` / `backlog` / `deprecated`), owner, dependencies, last touched. Example structure pre-filled. Feature Inventory is the doc that answers “what does this project actually do right now” without making anyone read the entire codebase.

7. Coding Conventions — `docs/conventions/CODING.md`

Opinionated TypeScript / React / Next.js defaults. No `any` types. No `console.log` in production code. No silent catches. Additive-only migrations. Direct `fetch` calls in server routes, not SDKs. The conventions doc is the document that gets cited every time an agent suggests something out of pattern — “see CODING.md §3” beats arguing in chat.

8. Security Gates — `docs/conventions/SECURITY.md`

Env var inventory, secrets handling (the 1Password convention — resolve at use-time, never write secrets to chat or files), auth flow expectations, RLS audit checklist if you’re on Supabase. Half a page. Cheap to write, expensive to skip.

9. Deployment Checklist — `docs/conventions/DEPLOYMENT.md`

Pre-deploy (typecheck, build, migrations applied?), deploy (commands, where), post-deploy (smoke test, edge functions redeployed if applicable, env vars present in target). Bullet list per stage. The checklist saves you the 11pm Tuesday outage where you forgot to redeploy an edge function and the queue worker has been quietly failing for six hours.

10. ADR template — `docs/adrs/`

The Architecture Decision Record format. Context, decision, consequences, status (`proposed` / `accepted` / `superseded`). Two example ADRs ship pre-filled — one stack choice, one architectural pivot. ADRs are for the big decisions that need pages of explanation; the Decision Log is for the rows that fit in one line.

11. README — `README.md`

The “what this repo is, what it isn’t, how to run, how agents should read it” template. Most READMEs read like documentation for humans. This one is calibrated for an agent and a returning operator both — short, ordered, link-heavy.

SECTION 4.

The three conventions that keep docs honest

Templates rot. The three conventions in this section are the difference between a doc set you maintain and a doc set that quietly turns into fiction. Other kits ship the templates and stop there.

Convention 1 — Supersession

Never delete a memory entry, decision, or ADR. Mark it superseded and link forward. The brain becomes an audit trail instead of a whiteboard.

The trap with file-based memory is the temptation to “clean up” stale entries by deleting them. You wake up six months in, look at a feedback memory that says “always use Bun” and another that says “Bun has FUSE issues — use Desktop Commander for installs,” and the natural move is to delete the older one. Don’t.

Mark the old entry with a `**Superseded by:**` line pointing at the new one. Leave the old entry in place. The new entry gets a `**Supersedes:**` line pointing back at the old one. Now an agent reading the file gets the full evolution — the original rule, the revision, and the reason for the change. The “why we changed our mind” stays auditable. The “what we believe now” is unambiguous.

```
---
name: bun-install-pattern
description: How to run bun install in this repo
type: feedback
status: superseded
superseded-by: bun-install-desktop-commander.md
---

Use `bun install` from workspace bash. Fast, no FUSE issues observed yet.
```

```
---
name: bun-install-desktop-commander
description: How to run bun install when sandboxed via FUSE
type: feedback
supersedes: bun-install-pattern.md
---
```

```
Use Desktop Commander's `start_process` for `bun install`. Workspace bash throws
`EPERM unlink` on FUSE-mounted projects. See incident 2026-04-22.
```

The supersession chain is two lines per entry. The audit value is the entire reason the brain doesn't drift.

Convention 2 — Cascade staleness

An optional `**Depends on:**` field plus a 50-line shell script that walks the dependency graph and surfaces dependents when an upstream changes.

The second trap is silent obsolescence — an upstream doc changes and three downstream docs are now wrong, but nobody knows because no one re-reads the downstream docs until something breaks. The cascade staleness convention solves it.

Every doc that depends on another adds a `**Depends on:**` field in its frontmatter listing the upstream paths. The kit ships `stale-audit.sh` (50 lines of bash, no dependencies) that walks the dependency graph. When you change an upstream doc, run the audit script — it prints the list of downstream docs that reference the changed file. You then decide which ones still hold and which need updating.

```
$ ./stale-audit.sh docs/conventions/CODING.md
[stale-audit] CODING.md changed at commit a7f3e21
[stale-audit] downstream dependents:
  docs/adrs/0003-react-query-defaults.md (depends on CODING.md §3)
  CLAUDE.md (references CODING.md as canonical conventions)
  docs/conventions/DEPLOYMENT.md (depends on CODING.md §5 build rules)
[stale-audit] review each one before next deploy.
```

The script is the discipline. Without it, "I'll remember to update the downstream docs" is the thing nobody remembers. With it, every upstream change has a 30-second cascade check.

Convention 3 — Lane Current_State

A per-lane auto-rolled `Current_State.md` that summarizes recent changes, open decisions, and what's loaded in working memory. A new agent reads one file and has 80% of the lane's context.

The third trap is context loss between sessions. An agent that did good work yesterday cannot read its own past — it can only read what you give it. If you give it the whole repo, it drowns. If you give it nothing, it relitigates. The Lane Current_State convention is the middle path.

Each working lane (a feature directory, a long-running branch, a sub-project) gets a `Current_State.md` at its root. The kit ships `lane-summary.sh` , a bash aggregator that walks

the lane directory, pulls the last N changelog entries, the open decisions, the active memory references, and writes them into Current_State.md. You run it at the end of each session. The next agent — yours, tomorrow, or a parallel dispatch lane — reads one file and gets the lane's full state.

```
$ ./lane-summary.sh src/features/billing
[lane-summary] writing src/features/billing/Current_State.md
[lane-summary] recent changes:
  2026-05-10 — wired Stripe Connect Express onboarding
  2026-05-08 — added webhook retry queue
[lane-summary] open decisions:
  3 — invoice email template tone
[lane-summary] active memory refs:
  feedback_stripe_idempotency.md
  reference_stripe_account.md
```

The file is short (under one screen by design). The agent reads it at lane start, knows where the lane is, and skips the "tell me what we've done so far" prelude that wastes the first ten messages of every session.

SECTION 5.

The 30-minute first pass

Don’t try to fill all eleven docs from scratch. Fill the five that the agent reads first, and backfill the others as the project surfaces them.

The 30-minute first pass is the smallest version of this kit that works. It produces a project the agent can be useful in starting today. The expansion happens organically.

1. **Minutes 0–10 — CLAUDE.md.** Project name, what it does, stack (React Query? Bun? Postgres?), one paragraph on “what this repo is and isn’t.” Link to the conventions doc even if it’s empty. Two pages max.
2. **Minutes 10–15 — AGENTS.md.** Copy the template verbatim. Read it. Edit two or three lines for your project — usually the “what to never touch” line is where personalization matters most.
3. **Minutes 15–20 — MEMORY.md scaffold.** Copy the index template. Drop one `user` entry describing yourself in three lines. Don’t fill in `feedback` or `project` entries yet — they’ll surface naturally.
4. **Minutes 20–25 — Decision Log.** The first three entries are usually free: “we picked Next.js because…”, “we picked Supabase because…”, “we picked Stripe because…”. Three rows. Done.
5. **Minutes 25–30 — README.md.** Two paragraphs. What this is. How to run it. Where the docs live.

The remaining six docs (Change Log, Feature Inventory, conventions, ADRs) fill in as the project asks. The first time a real decision needs a write-up, you’ll create the ADR file. The first time you ship a meaningful change, you’ll create the Change Log file. Trying to pre-fill them is the failure mode — you write fiction that the agent then treats as truth.

> THE 30-MINUTE TEST
>
> After 30 minutes, the agent should be able to read CLAUDE.md and answer: what is this project, what’s the stack, what are the standing rules, and what should I read next. If it can’t answer those four, your CLAUDE.md isn’t ready for the agent yet.

SECTION 6.

When to update each doc — and what triggers it

Documentation drift is not a discipline problem, it's a trigger problem. Each doc has a moment that should fire its update. Listing those moments is most of the work.

CLAUDE.MD	When the stack changes, when conventions change, when new agent rules are needed. Audit it monthly.
AGENTS.MD	When you onboard a new agent (Cursor → Codex → Manus) or your safety rails get a near-miss. Rare.
MEMORY.MD	Every time you give feedback to an agent that should persist ("don't do X, here's why"), file a `feedback` memory entry.
DECISION LOG	Every meaningful technical decision. The bar is: "would I want to know why this was chosen six months from now?" If yes, log it.
CHANGE LOG	Every shipped change that alters system behavior. Not every commit — every behavior change.
FEATURE INVENTORY	Whenever a feature ships, gets deprecated, or changes owner. Audit it quarterly.
CODING CONVENTIONS	When a pattern recurs in code review three times, lift it into the conventions doc.
SECURITY GATES	When a near-miss happens (auth bypass, leaked secret, RLS hole). Always.
DEPLOYMENT CHECKLIST	When a deploy goes sideways. Add the missed step to the checklist before the next deploy.
ADRS	When a decision is too big for a Decision Log row. Multi-paragraph rationale, alternatives explored, consequences enumerated.
README	When the run instructions change or a new contributor would be confused.

The conventions in §4 enforce themselves. The doc updates above don't, which is why the trigger list is critical. If you can't articulate the trigger for a doc, the doc will drift. If you can — and you build the trigger into your workflow (a pre-commit hook, a session-end ritual, a deploy checklist) — the docs stay current with almost no conscious effort.

SECTION 7.

What comes next

The kit is the artifacts. The community is the live system that keeps them current. The first is $15. The second is where the real compounding lives.

The Dev-Team Document Set is the static artifact bundle. Drop it in, fill it out, get a 5x improvement in agent consistency on day one. That's the kit's ROI.

The system around the kit — the patterns for evolving these artifacts as Claude, Cursor, Codex, and the MCP ecosystem all ship breaking changes — is what the AI Pocket Agency community is. It's $47/month, locked for life for the founding 50, at `aipocketagency.com`. Members get the live evolution of these docs, the multi-agent orchestration patterns (Dispatch Playbook kit ships free with community), the brain dashboard that surfaces all of this as readable signal, and a small group of operators who actually run this system. Member-only updates to this kit ship there before anywhere else.

If the kit unlocked something for you, that's where the next mile lives. Founding 50 closes when it closes. The $15 buys you the artifacts. The $47 buys you everything else.

Build something that doesn't slop.

— Chase